

Requirement Scope — Chat & Social Web Application

Living document · Version 0.3 · Last updated July 11, 2026

Status: Single-release build — everything in this document ships together (Section 22), no phased/future scope. No open questions remain (Section 23).

PART I — FOUNDATION

1. Overview

An interactive social web-application centered on chat — real-time one-to-one and group messaging combined with social features: profiles with granular privacy controls, friend connections, search and discovery, 24-hour status updates, live streaming, and an Instagram-style posts & feed with hashtags. Built with privacy, user-controlled visibility, and secure communication as core principles throughout, alongside basic content moderation and an admin analytics dashboard that never has access to private chat content.

2. Project Purpose

This is a learning / resume & portfolio project — not a commercial product. It exists to demonstrate skills, not to generate revenue.

3. Cost Constraint (Hard Requirement)

Every part of the implementation must be achievable on free tiers, with no paid services required at any point:

- No paid SMS/OTP providers (e.g. Twilio SMS) — use email-based or free alternatives for verification.
- No paid cloud infrastructure (e.g. AWS billed services) — use free-tier hosting (e.g. free static/Node hosts, free-tier databases).
- No paid third-party APIs (maps, push notification services, etc.) unless a free tier fully covers the use case.
- Any service used must have a permanently free tier suitable for a small/demo-scale app — not just a trial period.

- Native iOS/Android app store distribution is excluded — Apple's and Google's developer program fees are a real cost, so this stays a responsive web application only (no App Store/Play Store release).

4. Project Goals

The system should allow users to:

- Create and manage their accounts, with secure login and account recovery
- Create and manage profiles, with granular privacy controls
- Search and connect with other users, and become friends before messaging
- Chat one-to-one and in groups in real time, with attachments, replies, and edit/delete
- Share temporary status updates and go live to friends or everyone
- Post content with hashtags, and discover posts through search and a ranked feed
- Report abuse and be protected by moderation and blocking controls
- Trust that private data and chat content stay private, including from the application's own administrators

PART II — USERS & IDENTITY

5. User Types

5.1 Guest User

- Register an account
- Login
- Reset password using recovery options
- View public information where allowed

5.2 Registered User

- Manage their profile
- Search users
- Send friend requests
- Accept or reject friend requests
- Chat with accepted friends
- Manage privacy settings
- Block users
- Delete their account

5.3 Administrator

- Manage users

- Review reported accounts
- Handle abuse complaints
- Monitor system health
- Separate admin login, isolated from regular user accounts
- View an analytics dashboard (Section 18.1) — explicitly cannot read chat message content; access is limited to aggregate usage metrics and moderation reports

6. Authentication Requirements

6.1 Registration

Required: username, display name, password.

Optional: email address for account recovery.

- If email is provided, a confirmation step is required at signup — a verification link is emailed, and the address is only marked verified once the user clicks it
- Only gmail.com addresses are accepted; disposable/temporary email domains and any non-Gmail domain are rejected at signup
- Minimum age requirement at signup (e.g. 13+), checked against the birth date field where provided
- Username validation: allowed characters, min/max length, uniqueness check, and a reserved-word blocklist (e.g. "admin", "support")
- Password strength requirements enforced at signup (minimum length, complexity), with a live strength meter in the UI
- Consent checkbox at signup agreeing to the Terms of Service and Privacy Policy (Section 19) — required to complete registration
- CAPTCHA on signup and login to block automated/bot submissions (free tier, e.g. reCAPTCHA free tier or a self-hosted alternative)

6.2 Login

Users log in with username and password.

- If a verified email is on file, login also supports a magic-link flow: a one-time sign-in link is emailed to the user and opens the browser already logged in. Recommended over a manually-typed OTP — one tap, nothing to mistype, and it reuses the same email-verification channel already required elsewhere in this spec.

6.3 Password Management

- Change password
- Forgot password
- Reset password via email, if provided

6.4 Account Security

- Secure password storage (hashed, never plaintext)
- User sessions
- Logout functionality
- Multiple device sessions

6.5 Two-Factor Login & Session Management

- Optional two-factor login via email OTP for users with a verified email (free — same email channel as recovery/OTP login, Section 6.2)
- Settings view listing active logged-in sessions/devices, with the ability to revoke any of them remotely

6.6 New Device Login Verification

Applies only when the user has provided a recovery email (Section 6.1); adds a safeguard even if the account password is weak or compromised.

- System recognizes the device(s) a user has previously logged in from
- Login attempt from a new/unrecognized device pauses access and sends a confirmation email to the account's registered email
- UI shows a holding message, e.g.: "We've sent an email to abc@example.com — please check and confirm it's you"
- Login on the new device only completes after the user confirms via the emailed link ("Yes, it's me")
- If not confirmed, the new device is denied access to the account

6.7 First-Time Onboarding Tour

- On a user's first successful login, an automatic guided tour highlights the core features (profile setup, search, friend requests, chat, status/live, privacy settings)
- Tour is skippable and dismissible at any point
- Tour is shown only once per account (does not repeat on subsequent logins)

7. User Profile Requirements

Required fields: username, display name.

Optional fields: profile photo, bio/status, country, state, birth date, email.

Users should be able to:

- Update profile information
- Change profile photo
- Manage profile visibility

8. Privacy Requirements

Users control profile visibility, with three levels:

- **Public** — anyone can search and view profile information.
- **Friends Only** — only accepted friends can view profile details.
- **Private** — profile information is hidden from unknown users; only approved friends can view allowed information.

Users should also control:

- Who can send friend requests
- Whether email is visible
- Whether birth date is visible
- Online status & last-seen visibility — options: **Everyone, Friends Only, No One**

PART III — SOCIAL GRAPH & DISCOVERY

9. User Search Requirements

Users can be searched by username or display name.

Search rules:

- Public users can appear in search results
- Private users follow privacy rules
- Blocked users should not appear where applicable

10. Friend System Requirements

Before starting a conversation, users must become friends.

Friend request flow:

User A searches User B → User A sends friend request → User B accepts / rejects → if accepted, both become friends → chat becomes available

Users can:

- Send friend requests
- Accept requests
- Reject requests
- Cancel sent requests
- Remove friends

- Block users

10.1 Friend Suggestions

"People you may know" suggestions based on mutual friends.

10.2 Blocking Rules

When User A blocks User B, the block is one-directional but fully enforced against B from A's side:

- B cannot find A in search results (Section 9)
- B cannot view A's profile, even if A's profile is Public
- B cannot send A a friend request; any pending request between them is cancelled
- B cannot send A messages; existing conversation, if any, is locked from B's side
- A becomes effectively untraceable to B — no profile link, search result, or chat entry point remains available to B
- A can unblock B at any time, which restores normal visibility/interaction rules
- Blocking does not necessarily notify the blocked user

10.3 Invite Friends

Users can invite people outside the platform to join and connect via a shareable invite link.

- Generate a personal invite link (e.g. carries a referral/invite code tied to the inviting user)
- Copy link to clipboard
- Share directly via the device's installed apps (WhatsApp, Instagram, etc.) using the browser's native share option, where supported
- Opening the invite link takes a new user to registration; if the invite code maps to an existing user, a friend request/connection can be suggested after signup

PART IV — CONTENT: STATUS, LIVE & FEED

11. Status Updates

Users can post a temporary status (text/photo), similar to a "story," that automatically expires after 24 hours.

- Status auto-expires and is removed 24 hours after posting
- Visibility permission per status/profile setting: **Everyone** or **Friends Only**
- User must grant/select this permission when posting (or as a default profile setting)
- Only users covered by the chosen visibility can view the status while it is active
- Background music option — user can attach a track from a built-in royalty-free/free music library to a status (subject to the free-tier constraint in Section 3, no paid licensing)
- Image editing before posting — when attaching an image (to a status or as a chat image message), user can annotate it with markers/drawing tools and emoji stickers before sending

12. Live Streaming

Users can go live (broadcast real-time video) to an audience they choose.

Must use a free/self-hosted WebRTC approach (free STUN, self-hosted TURN) — no paid streaming/CDN service (e.g. paid tiers of Mux, AWS IVS).

- Visibility permission when going live: **Everyone** or **Friends Only**
- Live sessions are ephemeral — visible only while the broadcast is active, similar to Status Updates (Section 11)

12.1 Status & Live Rail

- A row of rounded profile-picture circles is shown at the top of the screen for friends/users with an active status or live stream, so it's easy to spot who has something to view
- A distinct ring/border style differentiates a plain **Status** (Section 11) from an active **Live** broadcast, so live is instantly recognizable at a glance
- Users currently live are sorted to the front of the rail, ahead of users with only a status

12.2 Active Users Indicator

- Shows a live count of currently active users, e.g. "X are active right now"
- Toggle to scope the count to friends only, e.g. "X friends are active now"
- Count respects each user's online-status visibility setting (Everyone / Friends Only / No One — Section 8); users set to "No One" are excluded from counts shown to others
- Count updates live as users connect/disconnect (ties to socket presence tracking, Section 21.1)

13. Posts & Feed

In addition to chat, users can post content (Instagram-style), separate from the ephemeral Status Updates (Section 11).

13.1 Creating Posts

- An upload/create-post action is available from the home screen (e.g. a prominent center control in the main navigation)
- Public-profile users can tag posts with hashtags

13.2 Hashtag Pages & Ranking

- Each hashtag has a page listing posts tagged with it
- Posts within a hashtag page are ranked by a combined score of likes, comments, and average views — an algorithmic sort similar to Instagram's, rather than strictly chronological

13.3 Post Visibility

- Public-profile users' posts can appear in hashtag pages/discovery
- Private-profile users' posts are visible only to their accepted friends — never in public hashtag discovery

13.4 Profile Stats

Every profile displays, Instagram-style:

- Number of posts
- Number of friends
- Number of pending sent friend requests

Uses the single friend-request relationship from Section 10 (no separate follower/following system) — chat unlocks once two users are accepted friends.

13.5 Post Interactions

- Like a post
- Comment on a post
- Save a post (bookmarked privately, doesn't notify the poster)
- Settings includes a "Posts I've Liked" view listing every post the user has liked
- Settings/profile includes a "Saved Posts" view listing bookmarked posts

13.6 Sharing Posts

- Share a post to another conversation within this web-application (target picker limited to friends, per Section 14.5's forwarding rule)
- Share a post outside the application (e.g. via the device's native share sheet, per Section 10.3) — the shared message/link includes an invite note along the lines of: "If the person you're sending this to isn't on the app yet, invite them — then enjoy your time and vibe!"

13.7 Explore / Discover Feed

A discover feed beyond hashtag pages, surfacing trending/suggested public posts. Feed loads via cursor-based infinite-scroll pagination.

13.8 Mentions

@username mentions in captions/comments, linking to the mentioned profile.

14. Messaging Requirements

14.1 Basic Messaging

- Start conversations with accepted friends
- Send text messages
- Receive messages
- View chat history
- Chat history and feed load via cursor-based infinite-scroll pagination (no full reload, fetches older items on scroll)
- Delivery & read status per message (sent / delivered / read), reflected as a status indicator in the chat UI
- Read-receipt opt-out setting — if a user turns off read receipts, they also stop seeing others' read receipts (mutual, like common chat apps)
- Message status updates pushed live over the socket connection and persisted in the database, so status is correct on reload or reconnect
- Unread message count/indicator per conversation, updated in real time via socket

14.2 Group Chat

- Create a group conversation with multiple friends
- Add/remove group members
- Send and receive text messages within the group
- Per-message, per-member delivery breakdown visible to the sender:
 - **Read** — member(s) who have read the message, each with a read timestamp
 - **Delivered / Notified** — member(s) whose device received the message but haven't read it yet
 - **Not yet notified** — member(s) not yet reached (e.g. offline, socket not connected)
- Status breakdown updates live via socket as each member's client acknowledges delivery/read (ties to Section 21.1)

14.3 Edit & Delete Messages

- Sender can edit a previously sent message; edited messages are marked "edited" to recipients
- Edit updates the message in the database and is pushed live to recipients via socket
- Sender can delete a message: **Delete for me** (removes from sender's view only) or **Delete for everyone** (removes from the conversation for all participants)
- Deletion is reflected live for all connected participants via socket; a placeholder (e.g. "message deleted") may remain in place of removed content
- Only the original sender can edit or delete-for-everyone their own messages

14.4 Additional Messaging Features

- Message reactions
- GIF support (sending/receiving animated GIFs in chat)
- Stickers
- Large single-emoji display — a message containing only one emoji renders enlarged, matching common chat-app behavior
- Voice and video calling — free/self-hosted WebRTC only (free STUN, self-hosted TURN), no paid calling API/service

14.5 Reply-to & Message Forwarding

- Reply to a specific earlier message: the reply shows a quoted reference (sender + snippet) of the original above it
- Tapping/clicking the quoted reference scrolls the chat to and highlights the original message, so the user can trace the reference back, similar to common chat apps
- Forward a message to another conversation — the forward target picker only lists accepted friends (Section 10), never arbitrary/searched users
- Forwarded messages are marked "Forwarded" to the recipient

14.6 Starred Messages

- User can star/unstar any message
- Starred messages are private to the user who starred them (starring is not visible to other participants)
- A dedicated "Starred Messages" view lists all starred messages across conversations, each linking back to its original chat/context

14.7 Link Safety

- URLs typed in a message are auto-detected and rendered as clickable links in the chat
- Clicking a link does not navigate immediately — an interstitial warning/confirmation dialog shows the destination URL and asks the user to confirm before leaving the app (e.g. "This link leads to an external site — do you trust it?")
- User can cancel and stay in-app, or proceed to open the link in a new tab

14.8 Attachment Picker

- An attach/pin option in the chat composer opens a picker with: Document, Image, Video, Audio, Camera (capture directly)
- All attachment types listed must work end-to-end (upload, delivery, preview/playback) — not just present as a UI option
- Every uploaded file, of any type, is capped at 10 MB; oversized files are rejected with a clear error before upload starts

- Images are compressed client-side before upload to stay under the cap and save bandwidth/storage

14.9 Appearance & Theme

- User can change app theme/layout (e.g. light/dark mode, and/or a choice of accent color or chat wallpaper)
- Theme preference saved per user account

14.10 Typing Indicator

Shows "[user] is typing..." live via socket while the other participant is composing a message.

14.11 Conversation Management

- Pin a conversation to the top of the chat list
- Mute notifications for a conversation
- Archive a conversation (hides it from the main list without deleting)
- Draft messages auto-saved per conversation if the user navigates away mid-typing, and restored when they return

14.12 In-Chat Search

Search within a conversation's message history by keyword. Since messages are encrypted at rest (Section 20) and not readable server-side even via direct database access, this cannot be a plain server-side database full-text search. Recommended approach: search runs client-side over the conversation history already decrypted and loaded into the chat UI — no server ever sees plaintext content to index.

15. Chat Privacy Rules

- Users cannot directly message unknown users
- Private profiles cannot be viewed by strangers
- Blocked users cannot communicate
- Only approved friendships allow chatting

PART VI — ACCOUNT, TRUST & SAFETY

16. Account Management

Users can update account information, change password, log out, and delete their account.

Account deletion options:

- **Temporary Deactivation** — account and profile become invisible to everyone, including existing friends (hidden from search, profile view, and chat); simply logging back in restores the account automatically to its prior state

- **Permanent Deletion** — account data is soft-deleted in the database (not hard-erased); it is not restored by a normal login — the user must go through a separate account-restoration/sign-in confirmation flow to reclaim it

16.1 Post-Login Menu & Settings

Once logged in, the user has access to:

- **Logout**
- **Settings**, containing:
 - Profile info — display picture, username, display name, bio, and other self-information fields (Section 7)
 - Account & privacy — visibility levels, who can send friend requests, email/birth date visibility, online/last-seen visibility, status visibility (Section 8)
 - Security — change password, connected/recognized devices, 2FA, active sessions (Sections 6.5–6.6)
 - Appearance — theme/layout preference (Section 14.9)
 - Account deletion

16.2 Data Export

User can request a downloadable export of their own account data (profile, posts, messages/chat backup).

17. Notifications

- Friend request received
- Friend request accepted
- New message received
- Account activity alerts
- Delivered as push/browser notifications, in addition to in-app notifications

18. Content Moderation & Reporting

- Report a post, message, or profile for abuse/inappropriate content
- Reports queue for review by the Administrator role (Section 5.3)

18.1 Admin Analytics Dashboard

Built on the self-hosted usage analytics from Section 21 — no paid analytics/BI service.

- Active users right now, daily/weekly active user counts
- Total registered users and growth over time
- Site visits/traffic over time
- Graph/chart visualizations of the above metrics (e.g. line/bar charts over a selectable date range)

- Admin access is strictly limited to aggregate metrics and moderation reports (Section 18) — admin cannot open or read any user's private chat content, consistent with the encryption-at-rest requirement (Section 20)

19. Legal

- Terms of Service and Privacy Policy pages, linked from signup and account settings
- Explicit consent checkbox at signup for data processing, referencing the Privacy Policy (ties to Section 6.1)

PART VII — NON-FUNCTIONAL & TECHNICAL

20. Non-Functional Requirements

Security

- Password hashing
- Secure authentication
- Access control
- Data privacy
- Messages stored encrypted at rest — not stored or readable as plain text by anyone, including via direct database access, to respect chat privacy. Only the sender/recipient(s) can decrypt (application-level encryption, keyed per conversation) — this is the recommended approach and is now the resolved design; see Section 14.12 for how in-chat search works within this constraint.
- Rate limiting on APIs and socket events (e.g. message sending, login attempts, friend requests) to prevent spam/flood loops and abuse — e.g. a cap of 20 outgoing pending friend requests per user at a time
- Standard web-security protections: input validation/sanitization, protection against SQL/NoSQL injection, XSS, and CSRF
- Basic brute-force protection on login (e.g. logout/backoff after repeated failed attempts)
- All traffic served over HTTPS/WSS (encrypted in transit), using the free TLS certificates most free-tier hosts provide (e.g. Let's Encrypt)
- Basic profanity/spam filtering on messages and posts using an open-source wordlist/library — no paid moderation API
- Security audit log of sensitive account events (login, password change, new-device confirmation) viewable by the account owner
- No promise of screenshot/copy prevention — not reliably achievable on the web and not claimed as a feature

Performance

- Fast user search

- Fast message delivery
- Efficient chat loading
- API optimization: paginate all list endpoints (feed, search, chat history — ties to Section 14.1/13.7 infinite scroll), avoid over-fetching (return only fields the view needs), and cache frequently-read data (e.g. profile info) at the application layer using free in-process/in-memory caching — no paid caching service (e.g. paid Redis tier)
- Database indexes on frequently queried fields (username, hashtags, message timestamps) to keep queries fast as data grows
- Debounce/throttle client-side calls that fire on typing (e.g. search-as-you-type) to reduce redundant API load

Scalability

- Not targeting large-scale growth — designed to run comfortably within free-tier hosting/database/storage limits (Section 3), consistent with this being a learning/portfolio project

UX Polish

- Helpful empty states for new users (empty feed, no friends yet, no messages) instead of a blank screen
- Loading skeletons (content-shaped placeholders) instead of blank screens or spinner-only states — applied consistently across feed, chat, profile, and search; this is a first-class UX requirement, not an afterthought
- Undo/confirmation snackbar after destructive actions (delete message, remove friend, delete post)
- Keyboard accessibility throughout — logical tab order, visible focus states
- Custom-styled 404 and error pages, consistent with the app's design

21. Technical Constraints

Hosting/infra choices are subject to the free-tier-only rule in Section 3; the specific provider is an implementation detail, not an open scope question.

- Offline-first caching via a service worker, so recently viewed chats/feed remain visible without network (free, browser-native)
- Basic self-hosted usage analytics (simple event counts in the app's own database) instead of a paid analytics tool
- Sitemap/robots.txt configuration so public profiles/posts are indexable (or excluded) by search engines as desired
- Health-check/status endpoint for uptime monitoring
- Scheduled/automated database backups (free-tier host snapshot, or a self-written export job)
- API responses expose rate-limit headers to clients (pairs with the rate limiting in Section 20)
- Environment-based configuration (dev/prod) — no secrets hardcoded in source
- Seed/demo data script for local development and for demoing the project

21.1 Real-Time Messaging & Socket Management

Chat must be delivered in real time using a persistent connection (e.g. WebSocket), not polling, and must run entirely on free-tier infrastructure (Section 3) — no paid managed real-time/socket service (e.g. paid tiers of Pusher, Ably).

- Establish a socket connection on login; close it on logout/disconnect
- Map each connected socket to the authenticated user (user ↔ socket session tracking)
- Delivery of a message only to sockets belonging to friends (respecting the friend/block rules in Section 10 and the chat privacy rules in Section 15)
- Handle reconnection after network drop without losing session
- Online/offline presence & last-seen tracking, shown per the user's visibility setting (Everyone / Friends Only / No One — Section 8)
- Support multiple sockets per user for multiple device sessions (ties to Section 6.4)
- Graceful fallback if the socket connection fails (e.g. queue message, retry)
- Message read/delivered status changes are written to the database as the source of truth and broadcast over the socket to update the sender's UI live

21.2 Message Ordering & Reliability

- Messages must arrive and render in the same order they were sent (e.g. sending 1, 2, 3, 4 must be received/displayed as 1, 2, 3, 4 — never reordered), even across reconnects or slow network
- Each message gets a unique ID and sequence marker so duplicate delivery (e.g. after a retry) can be detected and ignored (idempotent handling)
- Messages sent while offline or mid-connection-drop are queued locally and auto-retried once the connection recovers, rather than silently lost
- Sender sees a clear pending/failed state for a message stuck due to network issues, with a manual retry option
- Database remains the single source of truth; on reconnect the client reconciles/re-syncs its view against the server rather than trusting only what arrived over the socket

PART VIII — SCOPE

22. Release Scope

This is a single-release build — everything described in this document ships together, with no phased/future-scope split. Nothing in scope requires a paid service (Section 3); the one item removed on cost grounds is native iOS/Android app store distribution (Section 3) — the product remains a responsive web application.

Feature summary

- Accounts: registration, login (password + magic link), password management, 2FA, session management, new-device email confirmation, onboarding tour (Section 6)

- Profile & privacy: profile fields, visibility levels, granular privacy controls (Sections 7–8)
- Social graph: search, friend requests, suggestions, blocking, invites (Sections 9–10)
- Content: 24-hour status updates, live streaming, posts & feed with hashtags/likes/comments/saves/sharing (Sections 11–13)
- Messaging: one-to-one and group chat, attachments, edit/delete, reply/forward, starred messages, link safety, themes, typing indicator, conversation management, in-chat search (Section 14)
- Chat privacy rules (Section 15)
- Account management, settings, data export (Section 16)
- Notifications (Section 17)
- Content moderation, reporting, admin analytics dashboard (Section 18)
- Legal pages and consent (Section 19)
- Security, performance, and UX polish requirements (Section 20)
- Technical constraints: socket/real-time infrastructure, message ordering and reliability (Section 21)

23. Open Questions

None remaining. Prior open items are now resolved: login uses magic link (Section 6.2); in-chat search runs client-side over decrypted history to preserve encryption-at-rest (Sections 14.12 & 20); every feature in this document is in scope for the single release (Section 22).